

SMART DOCUMENT

ANALYST

1.	Project Overview	3
2.	System Architecture	4
3.	Agent Descriptions	5
4.	Tools and Their Schemas	6
5.	Agent Workflow	7
6.	Deep Learning Model	8
7.	Ollama LLM Integration	9
8.	Edge Case Testing	10
9.	Error Handling and Logging	11
10.	Project Structure and Setup	12

Project Type:	Integrated Project — Building Multi-Agent AI Systems
Technology:	Python · PyTorch · Ollama (phi3:latest) · TF-IDF
System Type:	Multi-Agent AI Workflow with Human-in-the-Loop
Document Types:	CV Invoice Report
Date:	May 2026

REALISED BY

Yassine Oukessou · Douae Jellal
Mohamed El Mehdi Saidi · Salma Abarkane

1. Project Overview

Smart Document Analyst is a multi-agent AI system designed to automatically classify, extract information from, and summarize business documents. It was developed as part of an Integrated Project focused on building real-world Multi-Agent AI Systems using modern deep learning and natural language processing techniques.

The system accepts raw text documents through a simple command-line interface and runs them through a pipeline of four coordinated agents. Each agent has a clear role, and together they produce a structured analysis that the user can approve before a final report is generated.

Key Features

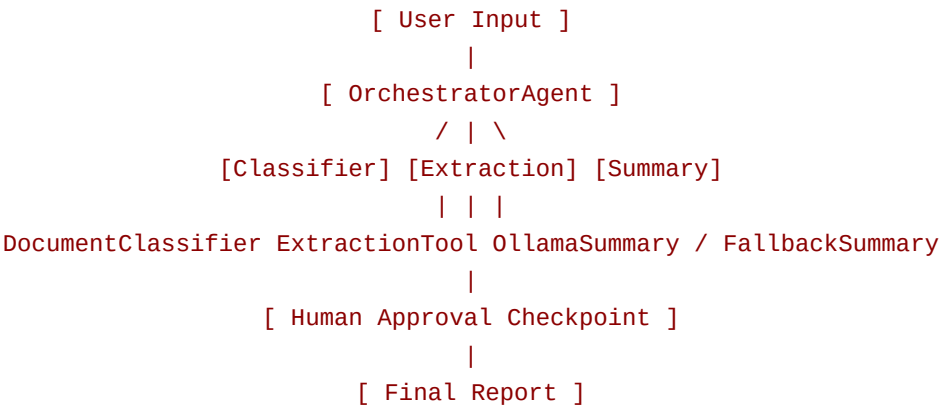
Document Classification	Automatically identifies whether a document is a CV, Invoice, or Report using a trained PyTorch neural network.
Information Extraction	Pulls out structured fields such as dates, amounts, email addresses, and keywords depending on the document type.
AI-Powered Summarization	Uses a locally running Ollama LLM (phi3:latest) to generate natural-language summaries, with a rule-based fallback.
Human-in-the-Loop	A mandatory approval checkpoint allows the user to review and accept or reject the analysis before finalisation.
JSON Logging	Every agent action is recorded with a timestamp in a structured JSON log file for traceability.
Robust Error Handling	The system gracefully handles empty inputs, missing files, ambiguous documents, and tool failures.

2. System Architecture

The system follows a sequential multi-agent architecture controlled by a central Orchestrator. Each specialist agent has a single well-defined responsibility and communicates only through the Orchestrator, which maintains the shared workflow state.

Agents	1 Orchestrator + 3 Specialist Agents (4 total)
Tools	4 tools (Classifier, Extraction, OllamaSummary, FallbackSummary)
Human Checkpoint	1 mandatory approval step before report finalisation
Communication	Sequential — each agent passes results to the next via the Orchestrator
Logging	JSON-formatted logs stored in logs/agent_logs.json
Error Handling	Fallback mechanisms and graceful degradation at every stage

Architecture Diagram



The Orchestrator is the only agent that holds global state. Specialist agents are stateless — they receive inputs, run their tool, and return structured outputs.

3. Agent Descriptions

OrchestratorAgent

Controls the entire workflow. It receives the raw document, passes it to each specialist agent in order, collects results, stores the workflow state, and presents the final output to the user for approval. It is the only agent with visibility into the full pipeline.

- Receives raw document text from the user
- Dispatches text to ClassifierAgent
- Sends classification + text to ExtractionAgent
- Sends all gathered data to SummaryAgent
- Presents results and requests human approval
- Stores the complete workflow state

ClassifierAgent

Classifies the document into one of three supported categories: CV, Invoice, or Report. It uses the DocumentClassifierTool which internally runs the trained PyTorch neural network.

- Input: raw document text
- Output: category, confidence score, per-class probabilities
- Tool used: DocumentClassifierTool

ExtractionAgent

Extracts structured information from the document based on the predicted category. Different fields are targeted for each document type — for example, an invoice will trigger extraction of amounts and client details, while a CV will focus on skills and dates.

- Input: raw text + predicted category
- Output: extracted_fields, dates, amounts, emails, keywords
- Tool used: ExtractionTool

SummaryAgent

Generates a natural-language summary using the Ollama LLM (phi3:latest). If Ollama is unavailable or returns an error, the agent automatically falls back to a rule-based summary tool to ensure the workflow always completes.

- Input: raw text + category + extracted information
 - Output: summary, word_count, key_points
 - Primary tool: OllamaSummaryTool
 - Fallback tool: SummaryTool (rule-based)
-

4. Tools and Their Schemas

Each agent uses one or more tools to perform its task. Tools are discrete, reusable components with clearly defined input and output schemas. The following section describes each tool.

DocumentClassifierTool

Loads the trained PyTorch model and TF-IDF vectorizer to classify input text into one of the three document categories.

Input schema:

```
Input: { "text": "string" }
```

Output schema:

```
{ "category": "cv | invoice | report", "confidence": 0.0, "probabilities": { "cv": 0.0, "invoice": 0.0, "report": 0.0 } }
```

ExtractionTool

Applies category-specific extraction rules. Different regex and keyword patterns are used depending on whether the document is a CV, Invoice, or Report.

Input schema:

```
{ "text": "string", "category": "cv | invoice | report" }
```

Output schema:

```
{ "category": "...", "extracted_fields": {}, "dates": [], "amounts": [], "emails": [], "keywords": [] }
```

OllamaSummaryTool

Sends a prompt to the locally running Ollama API (phi3:latest model) and returns a natural-language summary with key points.

Input schema:

```
{ "text": "string", "category": "cv | invoice | report", "extracted_info": {} }
```

Output schema:

```
{ "category": "...", "summary": "...", "word_count": 0, "key_points": [] }
```

SummaryTool (Fallback)

Rule-based fallback that constructs a summary from the extracted fields without requiring an LLM. Activated automatically when Ollama is unavailable.

Input schema:

```
{ "text": "string", "category": "cv | invoice | report", "extracted_info": {} }
```

Output schema:

```
{ "category": "...", "summary": "...", "word_count": 0, "key_points": [] }
```

5. Agent Workflow

The workflow is linear and sequential. Each step depends on the output of the previous one. The Orchestrator controls the flow and passes data between agents.

Step 1	User Input	The user provides raw document text through the CLI.
Step 2	Classification	ClassifierAgent uses the PyTorch model to identify the document category and confidence.
Step 3	Extraction	ExtractionAgent pulls structured data from the text based on the predicted category.
Step 4	Summarisation	SummaryAgent calls Ollama (phi3:latest) to produce a natural-language summary and key points.
Step 5	Fallback	If Ollama is unavailable, the rule-based SummaryTool is used automatically instead.
Step 6	Human Approval	The Orchestrator presents all results to the user, who must explicitly approve or reject.
Step 7	Final Report	If approved, the full analysis is saved as a final report. If rejected, the workflow ends without saving.

The human-in-the-loop checkpoint (Step 6) is mandatory and cannot be skipped. This design ensures that no analysis is finalised without explicit user confirmation, making the system suitable for real-world document processing where accuracy is important.

6. Deep Learning Model

The classification component is powered by a PyTorch feedforward neural network trained on a labelled dataset of business documents. The model uses TF-IDF feature extraction to convert raw text into numerical vectors before classification.

Model Architecture

Layer	Details
Input	TF-IDF feature vector (1,000 features, unigrams + bigrams)
Hidden Layer 1	Dense(256) + ReLU activation + Dropout(0.3)
Hidden Layer 2	Dense(128) + ReLU activation + Dropout(0.3)
Output Layer	Dense(num_classes) + Softmax
Output	Category: CV Invoice Report

Training Configuration

Parameter	Value
Optimiser	Adam
Loss Function	CrossEntropyLoss
Early Stopping	Patience of 10 epochs
TF-IDF Features	1,000 features (unigrams + bigrams)
Evaluation	Accuracy + Confusion Matrix + Classification Report
Model Files	document_classifier.pt, vectorizer.pkl, label_encoder.pkl

Evaluation results are automatically saved to model/evaluation_results.json after training. This file contains the accuracy score, confusion matrix, and a full classification report for each document class.

7. Ollama LLM Integration

The SummaryAgent integrates with a locally running Ollama instance to generate natural-language document summaries. Ollama allows large language models to run on local hardware without sending data to external APIs, which is important for document privacy.

Model Used	phi3:latest
Deployment	Local — runs on the user's machine via Ollama
Trigger	Called by OllamaSummaryTool during the summarisation step
Fallback	If Ollama is unavailable or returns an error, SummaryTool is used
Verification	Run 'ollama list' to confirm the model is available before starting

Fallback Mechanism

The SummaryAgent is designed to always succeed. If the Ollama API call fails for any reason — for example because Ollama is not running, the model is not installed, or a network error occurs — the agent automatically switches to the rule-based SummaryTool. This guarantees that the workflow completes even in offline or restricted environments.

This two-tier design (LLM preferred, rule-based fallback) makes the system both powerful and resilient. It demonstrates a practical approach to integrating LLMs in production systems where availability cannot always be guaranteed.

8. Edge Case Testing

The system was tested against a set of deliberately difficult or unusual inputs to verify robustness. The goal was to confirm that the system never crashes on invalid input and always returns a structurally valid response, even if the content is imperfect.

#	Case	Input Sample	Expected Behaviour
1	Empty Input	""	System rejects the input, returns a clear error message. No crash.
2	Whitespace-only Input	" "	Input is rejected gracefully. No crash.
3	Very Short Input	"Invoice"	Classification still runs. Low confidence is acceptable. No crash.
4	Ambiguous Content	"John Smith submitted an invoice for consulting..."	System picks one category. Extraction and summary still complete. Output is valid.
5	Invoice Without Amount	"Invoice #9001 for design services. Client: Nova."	Partial extraction completes. Summary is generated. Missing amount does not crash the system.
6	CV Without Name	"Experienced Python developer with 4 years in ML."	Classification works. Fallback values are used where fields are missing.
7	Report Without Metrics	"Quarterly report on internal process improvements."	Text is summarised. Missing financial data does not cause a failure.
8	Off-Domain Text	"The weather is pleasant today."	Classifier assigns the most likely category. Low confidence is expected. Output remains valid.

Edge case tests can be executed by running `tests/run_edge_case_checks.py`. Results are saved automatically to `logs/edge_case_results.json` for review.

9. Error Handling and Logging

Error Handling

The system handles errors at every stage of the pipeline. Rather than crashing on unexpected input or tool failure, it returns meaningful messages and uses fallback mechanisms to keep the workflow running.

Empty or whitespace input	Rejected at entry. User receives a clear error message.
Missing model files	System detects missing .pt or .pkl files and prompts the user to retrain.
Invalid classification output	Validated before passing to the next agent. Unexpected values are caught.
Ollama API failure	SummaryAgent automatically switches to the rule-based fallback tool.
File I/O errors	Caught and reported without terminating the main application.
Runtime classification errors	Caught per-agent; the Orchestrator is informed and can decide how to proceed.

JSON Logging

Every important action performed by an agent is recorded in a structured JSON log file. Each log entry includes the timestamp, agent name, action performed, and relevant details.

```
{ "timestamp": "2024-01-15T10:30:00", "agent": "ClassifierAgent", "action":  
  "classifying_document", "details": { "text_length": 150 } }
```

Logs are stored in logs/agent_logs.json and can be viewed directly from the CLI menu using option 3 (View logs). This makes the system fully auditable.

10. Project Structure and Setup

Project Directory Structure

```
smart_document_analyst/ |-- app/ | |-- agents.py | |-- logger.py | |-- main.py | `--
tools.py |-- data/ | `-- dataset.csv |-- logs/ | `-- agent_logs.json |-- model/ | |--
document_classifier.pt | |-- document_classifier_label_encoder.pkl | |--
document_classifier_vectorizer.pkl | `-- train_model.py |-- ARCHITECTURE.md |--
final_report.txt |-- README.md `-- requirements.txt
```

Installation Steps

1. Navigate to project folder:

```
cd smart_document_analyst
```

2. Install dependencies:

```
pip install -r requirements.txt
```

3. Verify Ollama model:

```
ollama list (must show phi3:latest)
```

4. Train the model (if needed):

```
python model/train_model.py
```

5. Run the application:

```
python app/main.py
```

CLI Menu Options

Option	Action
1	Analyze a document — enter text for full classification, extraction, and summarisation
2	Train the model — retrain the PyTorch classifier from the dataset
3	View logs — display the JSON agent activity log
4	Generate final report — create the output report file
5	Exit — close the application

Requirements Checklist

- ✓ Multi-agent system with 4 agents
- ✓ OrchestratorAgent coordinating all specialist agents
- ✓ At least 2 specialist agents (ClassifierAgent, ExtractionAgent, SummaryAgent)
- ✓ PyTorch deep learning model used for classification
- ✓ DL model exposed as a functional tool (DocumentClassifierTool)
- ✓ Ollama LLM integrated into the SummaryAgent (phi3:latest)
- ✓ Human-in-the-loop approval checkpoint
- ✓ JSON logging with timestamps for all agent actions
- ✓ Error handling at every pipeline stage
- ✓ Final report generation saved to file
- ✓ Simple and usable CLI application

✓ Model evaluation results saved to JSON (accuracy, confusion matrix, report)

Realised by: Yassine Oukessou | Douae Jellal | Mohamed El Mehdi Saidi | Salma Abarkane
Educational Project — Integrated Project: Building Multi-Agent AI Systems